

How Airbnb Runs Distributed Databases on Kubernetes at Scale



BYTEBYTEGO

OCT 01, 2025

246

3

15

Share

[Stop Agent Hallucinations with Project Rules \(Warp University\) \(Sponsored\).](#)



AI coding agents are here, but harnessing that power without agents going off the rails isn't always simple.

Warp's Project Rules solves that. Warp is the top-ranked coding agent (outperforming Claude Code, Cursor, and Codex on benchmarks) and is trusted by 700K+ developer

In this Warp University lesson, you'll learn how to use Project Rules to give agents reliable context, so they stop hallucinating and start shipping.

Disclaimer: The details in this post have been derived from the official documentation shared online by the Airbnb Engineering Team. All credit for the technical details goes to the Airbnb Engineering Team. The links to the original articles and sources are present in the reference section at the end of the post. We've attempted to analyze the details and provide our input about them. If you find any inaccuracies or omissions, please leave a comment, and we will do our best to fix them.

For years, companies relied on large, expensive standalone servers to run their databases. As traffic increased, the standard approach was to implement sharding, which involved dividing the data into smaller pieces and distributing them across multiple machines. While this worked for a time, it became a heavy burden. Maintaining shards, handling upgrades, and keeping the whole system flexible turned into a complex and expensive problem.

The last decade has seen the rise of distributed, horizontally scalable open-source SQL databases. These systems allow organizations to spread data across many smaller machines rather than relying on a single giant one.

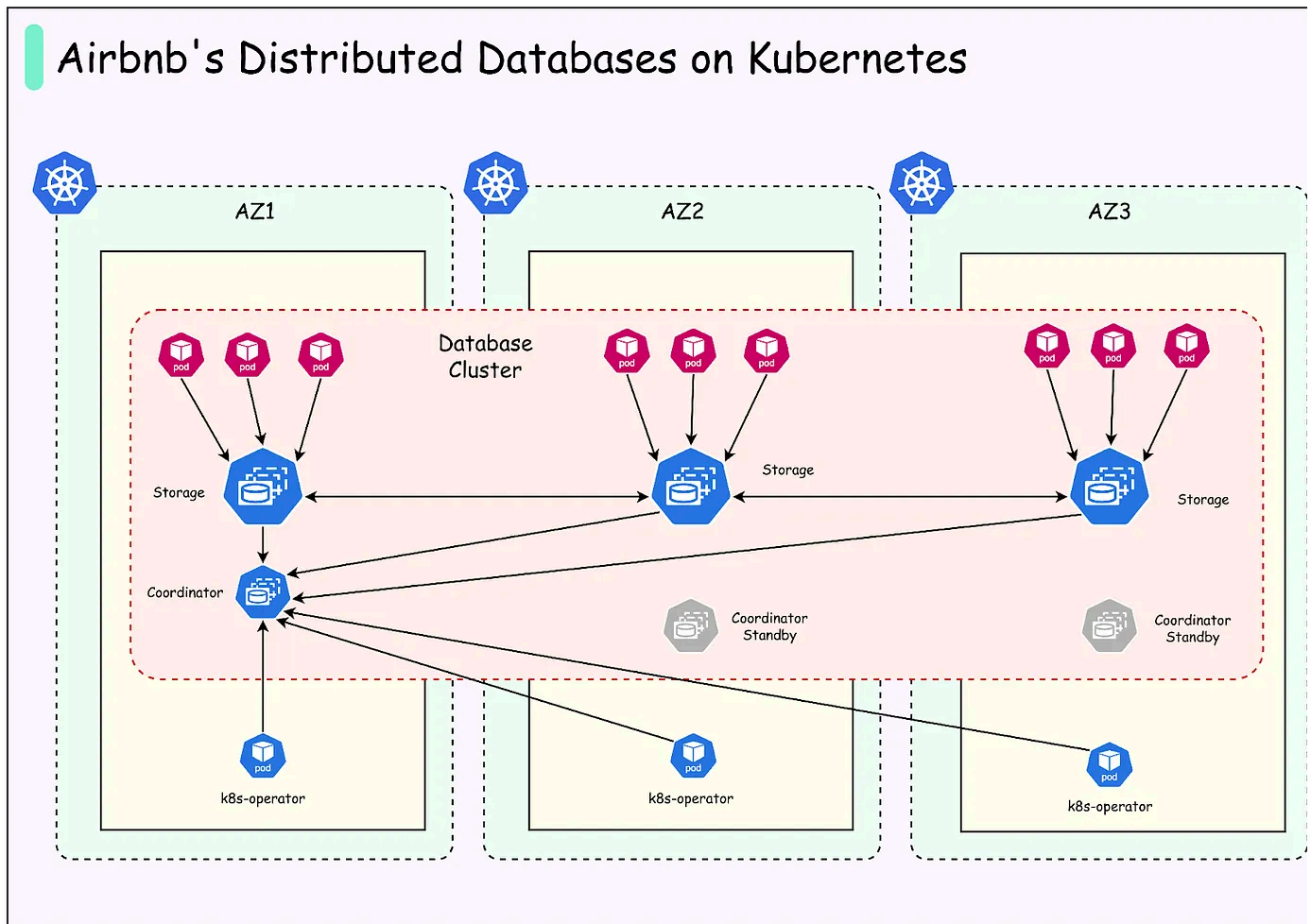
However, there's a catch: running such databases reliably in the cloud is far from simple. It's not just about spinning up more servers. You need to ensure strong consistency, high availability, and low latency, all without increasing costs. This balance has proven tricky for even the most advanced engineering teams.

This is where Airbnb's engineering team took an unusual path.

Instead of limiting a database cluster to one Kubernetes environment, they chose to deploy distributed database clusters across multiple Kubernetes clusters, each one mapped to a different AWS Availability Zone.

This is not a common design pattern. Most companies avoid it because of the added complexity. But Airbnb's engineers saw it as the best way to ensure reliability, reduce the impact of failures, and keep operations smooth.

In this article, we will look at how Airbnb implemented this design and the challenges they faced.



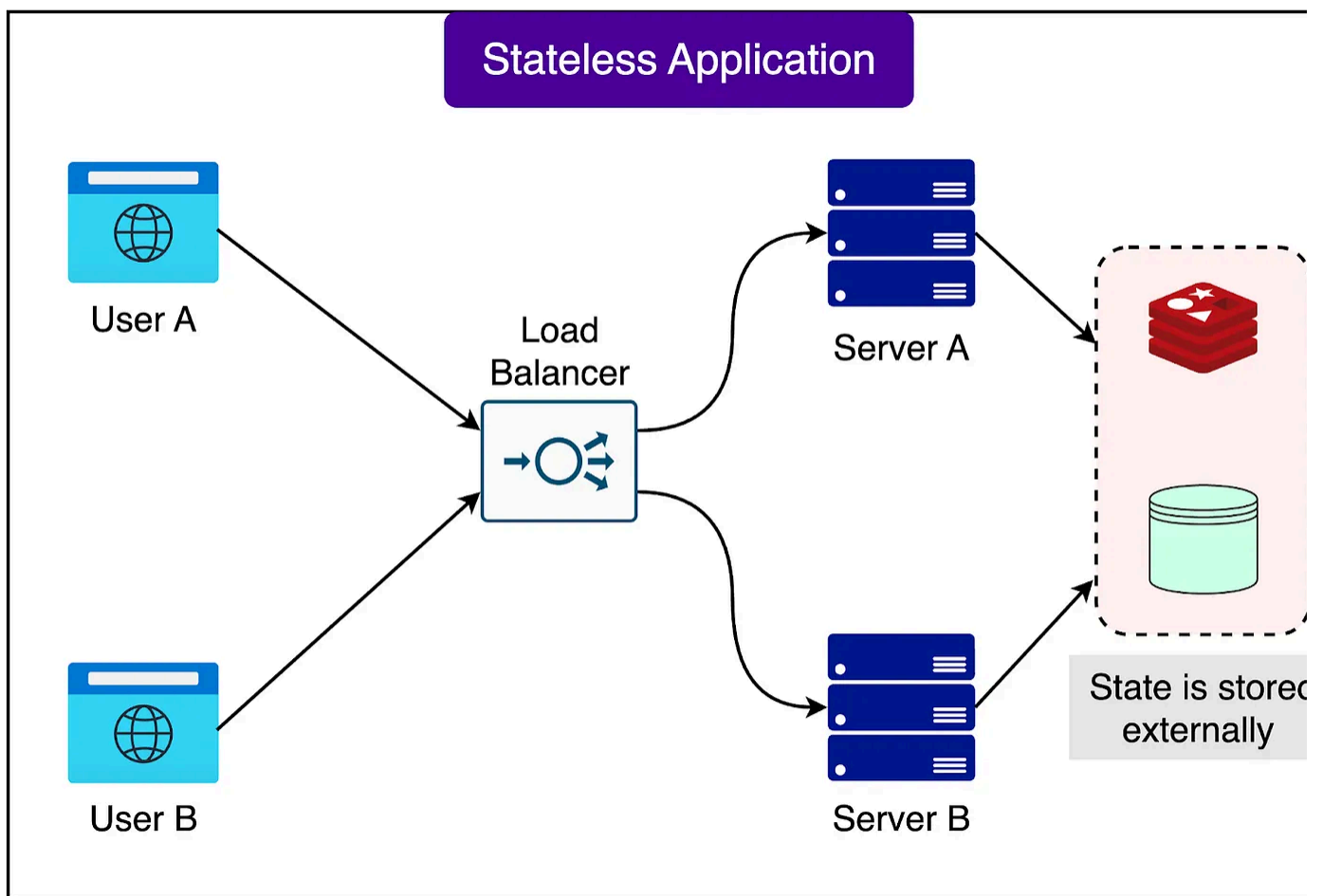
Help us Make ByteByteGo Newsletter Better

TL:DR: Take this 2-minute survey so I can learn more about who you are, what you do, and how I can improve ByteByteGo

Running Databases on Kubernetes

Kubernetes is very good at running stateless workloads. A workload is called stateless when it does not need to remember anything between requests. For example, a web server that simply returns HTML pages is stateless. If one server goes down, another can immediately take over, because there is no important memory or data tied to a single machine.

See the diagram below:



Databases, however, are stateful. They must keep track of data, store it reliably, and make sure that changes are not lost.

Running stateful systems on Kubernetes is harder because when a database node is replaced or restarted, the system must ensure the stored data is not corrupted or lost.

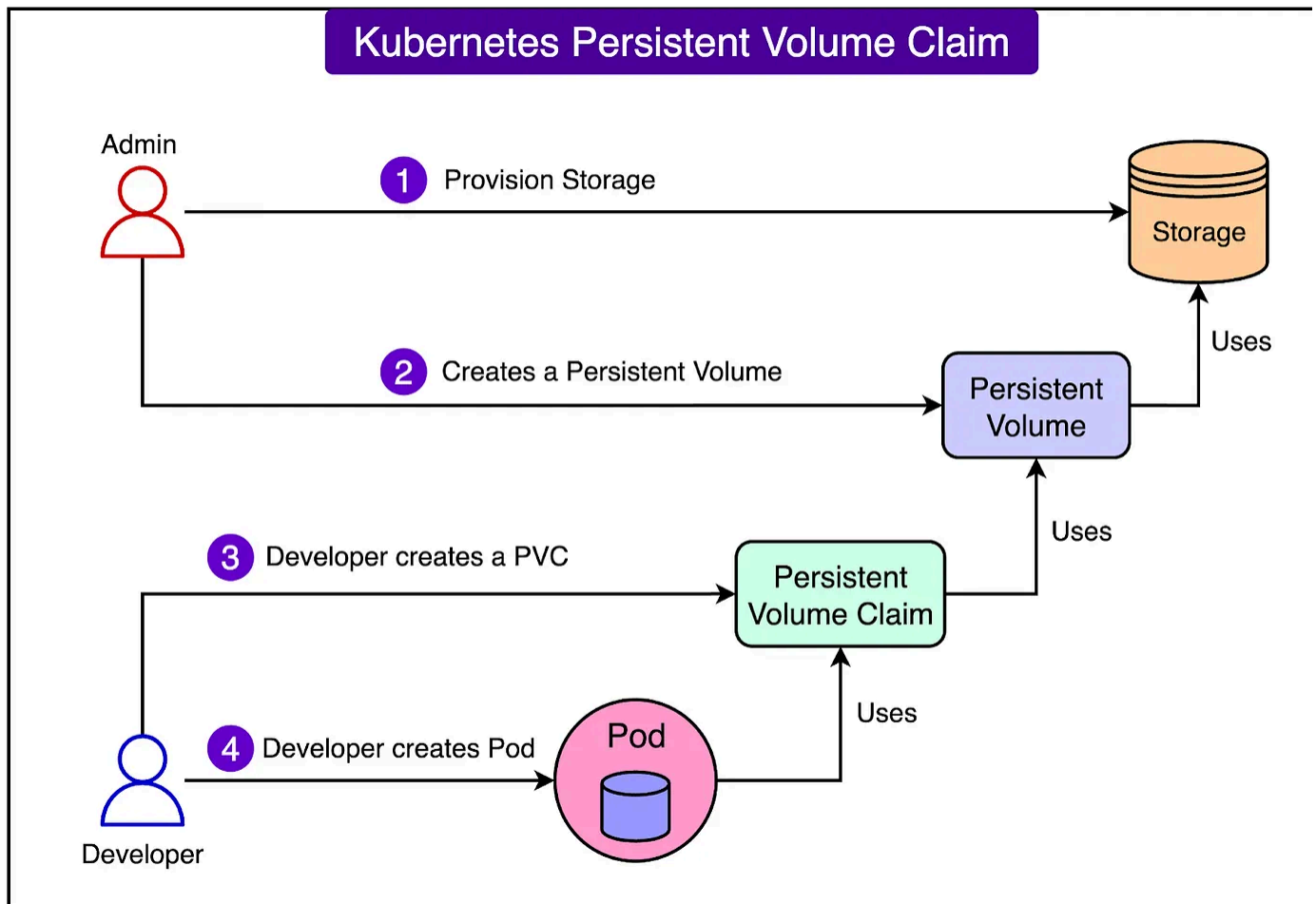
One of the biggest risks comes from node replacement. In a distributed database, data is stored across several nodes. To guarantee correctness, the cluster depends on a quorum. A quorum means that a majority of nodes (for example, 2 out of 3 or 3 out of 5) must agree on the current state of data. If too many nodes fail or get replaced at the wrong time, the quorum is lost and the database can stop serving requests.

Kubernetes, by default, does not understand how the data is spread across nodes, so it cannot prevent dangerous replacements.

The Airbnb engineering team solved this problem with a few smart techniques:

- **Using AWS EBS volumes:** Each database node stores its data on Amazon Elastic Block Store (EBS), which is a durable cloud storage system. If a node is terminated or replaced, the EBS volume can be detached from the old machine and quickly reattached to a new one. This avoids having to copy the entire dataset to a fresh node from scratch.
- **Persistent Volume Claims (PVCs):** In Kubernetes, a PVC is a way for an application to request storage that survives beyond the life of a single pod or container. Airbnb uses PVCs so that when a new node is created, Kubernetes automatically reattaches the existing EBS volume to it. This automation reduces the chance of human error and speeds up recovery.
- **Custom Kubernetes Operator:** An operator is like a “smart controller” that extends Kubernetes with application-specific logic. Airbnb built a custom operator for its distributed database. Its job is to carefully manage node replacements. It makes sure that when a new node comes online, it has fully synchronized with the cluster before another replacement happens. This process is called serializing node replacements, and it prevents multiple risky changes from happening at the same time.

See the diagram below that shows the concept of a PVC in Kubernetes:



By combining EBS volumes, PVCs, and a custom operator, Airbnb has found a way safely run a distributed database on Kubernetes. This allows them to benefit from Kubernetes automation without sacrificing the consistency or availability that a database requires.

Node Replacement Coordination

In any large system, database nodes will eventually need to be replaced. This can happen for several reasons, and Airbnb groups them into three categories:

- **Database-initiated events:** These are planned changes like configuration updates or version upgrades of the database software.

- **Proactive infrastructure events:** These happen when the cloud provider (in this case, AWS) retires old virtual machines or when engineers perform routine upgrades.
- **Unplanned failures:** These are unexpected issues, such as a hardware crash or node becoming unresponsive.

Each of these situations requires a different handling strategy.

Here's how Airbnb handles them:

- **Custom operator checks for planned events:** When the database itself needs a change, Airbnb's custom operator first checks that every node in the cluster is up and running. Only then does it allow the replacement of a pod. This prevents a situation where multiple unhealthy nodes are replaced at the same time, which could bring the cluster down.
- **Admission hooks for infrastructure events:** Kubernetes has a feature called an admission hook, which is a way to intercept or modify requests before they are carried out. Airbnb uses this to catch pod eviction requests that come from the infrastructure layer (like AWS retiring a VM). Instead of allowing the eviction right away, the hook blocks it and attaches a special annotation to the pod. The custom operator then notices the annotation and safely deletes the pod in a controlled way. This process ensures that these infrastructure-driven replacements happen in sequence with any database-driven replacements.
- **Unplanned failures:** If a machine suddenly dies, there is no way to coordinate in advance. However, Airbnb's operator makes sure that while a failed node is being replaced, no other planned replacements are allowed. This ensures that the system still maintains quorum and avoids further instability during recovery.

In short, Airbnb built a layered safety net. Their custom operator works with admission hooks to serialize planned replacements, while also pausing other maintenance when an unplanned failure is already in progress. This careful

coordination is what keeps the database cluster available even in the face of node churn.

Kubernetes Upgrades

Keeping Kubernetes up to date is important for security, stability, and access to new features. However, upgrades are particularly risky when running databases. Unlike stateless services, a database cannot simply restart without consequences, because it needs to protect the integrity and availability of stored data.

One of the main problems is with cloud-managed Kubernetes services. In these systems, when the control plane (the central component that manages all the worker nodes and scheduling) is upgraded, there is usually no way to roll it back if something goes wrong. This means a bad upgrade can lead to serious disruption, and the only option is to wait while the provider fixes it.

To reduce this risk, the Airbnb engineering team uses self-managed Kubernetes clusters. In a self-managed setup, engineers have full control over the cluster and can perform a rollback if an upgrade causes issues. This is safer, but it does not completely remove the risk. If a rollback is needed, the database could still experience downtime or instability until the rollback is finished.

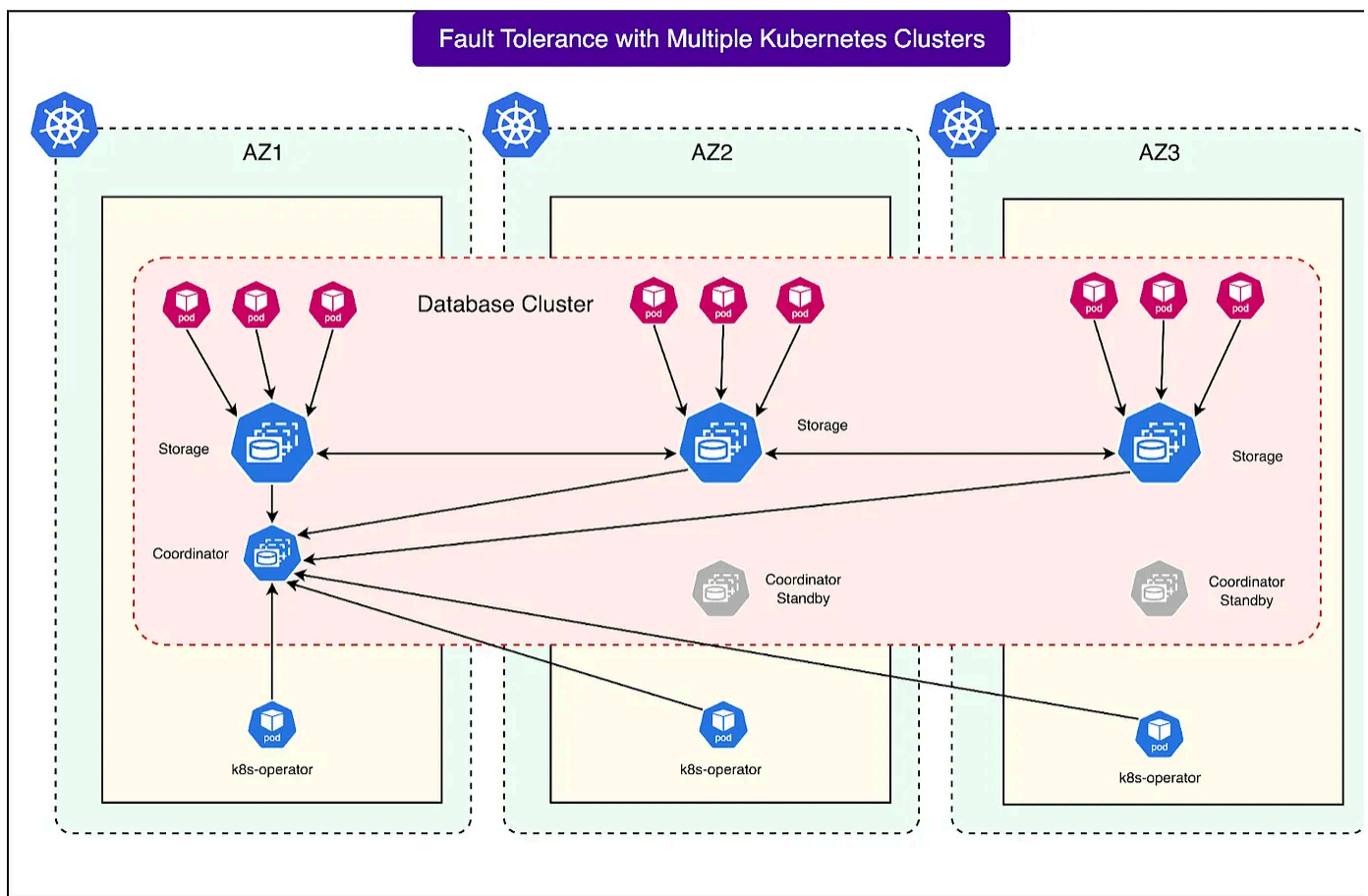
Airbnb's strategy is to rely on careful sequencing of upgrades and its multi-cluster deployment design. Instead of upgrading everything at once, they upgrade one cluster at a time, starting with a smaller or less critical cluster. Since each database spans multiple clusters across different availability zones, even if one cluster is temporarily unstable during an upgrade, the others continue serving traffic. This greatly reduces the blast radius of a bad upgrade and helps ensure that the overall system remains available.

Multi-Cluster Deployment for Fault Tolerance

One of the most important choices Airbnb made was to deploy each database across three separate Kubernetes clusters, with each cluster running in a different AWS Availability Zone (AZ).

An Availability Zone is like an independent data center within a region, with its own power, networking, and hardware. By spreading clusters across three AZs, Airbnb ensures that the failure of one zone does not take down the entire database.

See the diagram below:



This setup brings several benefits:

- **Fault isolation:** AWS designs its Availability Zones so that problems in one zone do not affect the others. For example, a networking outage in one AZ should not spill over into another. By aligning their clusters with AZs, Airbnb inherits this

isolation. If something goes wrong at the infrastructure level, the damage is contained to just one zone.

- **Reduced blast radius of deployments:** A “blast radius” refers to how far the impact of a failure spreads. Because Airbnb deploys databases across three clusters, a bad configuration change or bug introduced during a rollout will only affect one cluster. The other two continue serving queries, so the system as a whole stays available.
- **Canary deployments for safety:** Airbnb does not roll out changes everywhere at once. Instead, they first push new database configurations or software versions just one cluster in a single AZ. This acts like a canary test. If the change causes issues, they can catch it early before it spreads across all clusters.
- **Overprovisioning for resilience:** To prepare for the worst-case scenario, Airbnb deliberately runs more capacity than it needs. That way, even if an entire Availability Zone or a full Kubernetes cluster goes offline, the remaining clusters can still handle the full workload. This extra capacity ensures smooth operation under failure conditions.

By combining these practices, Airbnb created a database architecture that tolerates failures gracefully. During one incident, a faulty configuration abruptly terminated virtual machines of a specific type in Airbnb’s staging environment. This wiped out most of the query layer pods in that cluster. However, because the database was running in three clusters, the other two clusters continued operating normally. This prevented the problem from affecting users.

Leveraging AWS EBS

For storage, Airbnb chose to use Amazon Elastic Block Store (EBS). EBS is a block-level storage service that provides highly durable and reliable disks for cloud workloads. It plays a key role in making Airbnb’s database clusters both resilient and efficient.

EBS helps in the following manner:

Fast reattachment during node replacement: If a node goes down, its EBS volume be detached and quickly reattached to a new machine. This makes recovery much faster compared to copying data onto a fresh disk.

Higher durability compared to local disks: Local disks are tied to the machine they run on. If the machine fails, data can be lost. EBS, on the other hand, is built to survive machine failures, which provides much stronger guarantees for storing critical database information.

Reliable clusters with fewer replicas: Thanks to the durability of EBS, Airbnb can confidently run its database with just three replicas of the data. Without EBS-level durability, they might need more replicas to guard against data loss, which would increase costs and complexity.

The Latency Challenge

Despite its durability, EBS is not perfect. At times, it experiences tail latency spikes where a small fraction of requests take much longer than normal.

In Airbnb's case, the p99 latency (the 99th percentile of requests) sometimes reaches up to one full second. For a high-throughput database, even rare one-second delays can cause problems.

Some solutions Airbnb implemented are as follows:

- **Storage read timeout and retries:** The team added a session variable for storage read timeouts. If EBS stalls, the query can automatically retry on another node in the cluster instead of waiting indefinitely.
- **Replica reads for better distribution:** By default, the database retries requests only at the leader node, which can become a bottleneck. Airbnb changed this behavior to allow retries at both the leader and the replicas. The system prefers

the closest replica for performance reasons, which also avoids expensive cross-network traffic.

- **Stale reads for tolerant workloads:** For use cases that do not need the very latest data, Airbnb enabled stale reads. With this feature, replicas can serve queries independently without contacting the leader. If the leader happens to be experiencing an EBS latency spike, replicas can still handle the workload, keep the system responsive.

By combining these techniques, Airbnb reduced the impact of EBS latency spikes while maintaining the reliability benefits of EBS. The result is a system that can handle millions of queries per second without sacrificing consistency or performance for critical workloads.

Conclusion

Airbnb's engineering team set out to solve one of the hardest challenges in modern infrastructure: running a distributed database reliably on Kubernetes.

By carefully combining Kubernetes operators, multi-cluster deployments, AWS EBS, and advanced read strategies like replica reads and stale reads, they were able to build a system that is both highly available and cost-efficient.

This design is not just a theory. It is already running in production. Airbnb operates several clusters today, with its largest cluster reaching impressive numbers:

- 3 million queries per second (QPS) handled reliably
- 150 storage nodes working together
- Over 300 terabytes of data stored
- 4 million internal shards distribute the load
- 99.95 percent availability, meeting strict reliability requirements

With this setup, Airbnb has achieved the goals they were aiming for: high availability, low latency, scalability, and reduced maintenance costs. The system can survive node replacements, infrastructure failures, and even the loss of an entire availability zone without bringing down the service.

The key takeaway is that open-source distributed databases can succeed on Kubernetes when combined with thoughtful engineering. Airbnb's work shows that even though running stateful systems in Kubernetes is difficult, the benefits in automation, resilience, and cost efficiency make it worth the effort.

For other engineering teams, Airbnb's approach provides a roadmap: start small, use operators for safety, design for fault isolation across clusters, and build strategies to handle the edge cases like storage latency. With these practices, high availability in cloud becomes not just possible but sustainable at scale.

References:

- [Achieving High Availability with Distributed Database on Kubernetes at Airbnb](#)
- [Kubernetes Operator Pattern](#)

SPONSOR US

Get your product in front of more than 1,000,000 tech professionals.

Our newsletter puts your products and services directly in front of an audience that matters - hundreds of thousands of engineering leaders and senior engineers - who have influence over significant tech decisions and big purchases.

Space Fills Up Fast - Reserve Today

Ad spots typically sell out about 4 weeks in advance. To ensure your ad reaches this influential audience, reserve your space now by emailing sponsorship@bytebytego.com.



246 Likes • 15 Restacks

Discussion about this post

Comments Restacks



Write a comment...



dzen  Oct 1, 2025

How much cost savings from paying a large team to maintain this vs running off of an existing distributed database?

 LIKE (4)  REPLY



Helmar Oct 2, 2025

The architectural diagram shows 3 clusters. Each consist of 3 pods and storages. In order to there must be synchronization across clusters. How do they achieve that?

 LIKE (2)  REPLY



1 more comment...

© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture